

Computer Organization & Architecture Revision Sheet

This note is designed for rapid recall of instruction pipelining, execution speedup calculations, and cache memory structures.

[OS] 1. Instruction Pipelining clock cycle Calculations

Pipelining overlaps instruction execution to maximize CPU instruction throughput.

Core Formulas

Let: * N = Number of instructions to execute. * K = Number of pipeline stages. * T_n = Execution time of a single instruction on a **non-pipelined** processor. * T_p = Clock cycle time of the **pipelined** processor: $T_p = \max(\text{Stage Delays}) + \text{Buffer/Register Overhead}$ * SS = Speedup Ratio.

1. Total Pipelined Clock Cycles (No Hazards/Stalls)

$\text{Cycles}_{\text{ideal}} = K + N - 1$ * *Explanation:* The first instruction takes K cycles to exit the pipeline. Each of the remaining $N-1$ instructions takes exactly 1 cycle to exit.

2. Total Pipelined Clock Cycles (With Hazards/Stalls)

$\text{Cycles}_{\text{actual}} = (K + N - 1) + \text{Total Stall Cycles}$

3. Speedup Ratio (SS)

$SS = \frac{\text{Execution Time}_{\text{non-pipelined}}}{\text{Execution Time}_{\text{pipelined}}} = \frac{N \cdot T_n}{\text{Cycles}_{\text{actual}} \cdot T_p}$ * **Theoretical Maximum Speedup** (SS_{max}) under ideal conditions as $N \rightarrow \infty$: $SS_{\text{max}} = \frac{T_n}{T_p} \approx K$ (if $T_n = K \cdot T_p$)

2. Worked Pipeline Hazard Examples

These scenarios are frequently asked in NIELIT and GATE exams.

Example A: The 100-Instruction, 1-Stall Hazard Problem

- Scenario: A machine has 5 pipeline stages:
 - Fetch (9 ns), Decode (3 ns), Execute (7 ns), Memory (9 ns), Write-back (2 ns).
- Pipeline register overhead (latch delay) = 1 ns.
- A program with $N = 100$ instructions is run.
- Every **3rd instruction** requires a **1-cycle stall** before the Execute (X) stage.
- Goal: Calculate the total CPU execution time.

Step 1: Find the Pipelined Clock Cycle Time (T_p)

$T_p = \max(9, 3, 7, 9, 2) + 1 \text{ ns (overhead)} = 9 \text{ ns} + 1 \text{ ns} = 10 \text{ ns}$

Step 2: Calculate Ideal Cycles

$\text{Cycles}_{\text{ideal}} = K + N - 1 = 5 + 100 - 1 = 104 \text{ cycles}$

Step 3: Calculate Total Stall Cycles

Since every 3rd instruction stalls: $\text{Stalls} = \lfloor \frac{100}{3} \rfloor = 33 \text{ cycles}$

Step 4: Calculate Actual Cycles and Execution Time

$\text{Cycles}_{\text{actual}} = 104 + 33 = 137 \text{ cycles}$ $\text{Execution Time} = 137 \text{ cycles} \times 10 \text{ ns/cycle} = 1370 \text{ ns} = 1.37 \mu\text{s}$

Example B: Non-Pipeline to Pipeline Speedup

- **Scenario:** A non-pipeline system takes 50 ns to process a single task. The same task can be processed in a 6-segment pipeline with a clock cycle of 10 ns.
- **Goal:** Find the speedup ratio for $N = 100$ tasks, and find the maximum theoretical speedup.

Step 1: Calculate Non-Pipelined Time

$$T_{\text{non-pipelined}} = N \cdot T_n = 100 \cdot 50 \text{ ns} = 5000 \text{ ns}$$

Step 2: Calculate Pipelined Time

$$Cycles = K + N - 1 = 6 + 100 - 1 = 105 \text{ cycles}$$

$$T_{\text{pipelined}} = 105 \text{ cycles} \cdot 10 \text{ ns/cycle} = 1050 \text{ ns}$$

Step 3: Calculate Speedup Ratio (\$\$)

$$S = \frac{5000 \text{ ns}}{1050 \text{ ns}} \approx 4.76$$

Step 4: Maximum Theoretical Speedup

$$S_{\text{max}} = \frac{T_n}{T_p} = \frac{50 \text{ ns}}{10 \text{ ns}} = 5$$

3. Cache Memory Address Mapping division

When a memory address is issued by the CPU, it is divided into fields depending on the cache organization.

Let: * $\text{Physical Address Size} = A$ bits. * $\text{Cache Size} = C$ bytes. * $\text{Block Size / Line Size} = B = 2^b$ bytes $\implies$ Block Offset $= b$ bits. * Number of Cache Lines $= L = \frac{C}{B} = 2^l$ lines.

1. Direct Mapping

A memory block can map to exactly one cache line: $\text{line index} = (\text{Block Address}) \bmod L$.

$$\begin{array}{|c|c|c|} \hline \text{Tag (A-l-b bits)} & \text{Line Index (l bits)} & \text{Block Offset (b bits)} \\ \hline \end{array}$$

2. Fully Associative Mapping

A memory block can map to any line in the cache. No line index field is needed.

$$\begin{array}{|c|c|} \hline \text{Tag (A-b bits)} & \text{Block Offset (b bits)} \\ \hline \end{array}$$

3. K-Way Set-Associative Mapping

Cache lines are grouped into sets of size K . A memory block maps to exactly one set: $\text{set index} = (\text{Block Address}) \bmod S$. * Number of Sets $= S = \frac{L}{K} = 2^s$ sets $\implies$ Set Index $= s$ bits.

$$\begin{array}{|c|c|c|} \hline \text{Tag (A-s-b bits)} & \text{Set Index (s bits)} & \text{Block Offset (b bits)} \\ \hline \end{array}$$

COA Common Traps

- **Register Delay Overhead:** Always add the register/latch overhead to the max stage delay when calculating T_p . *Do not add it to the unpipelined T_n unless explicitly instructed!*
- **Stall Placement:** Read closely whether a hazard stalls a single instruction or multiple. Stalls are added as extra clock cycles directly to the total.
- **Byte vs. Word Addressable:** Always check if the memory is byte or word addressable before calculating block offset and tag bits!